

AIGIS-IDS-SCADA-IEC104

AIGIS is an Intrusion Detection System based on Machine Learning (ML) and Deep Learning (DL) algorithms for SCADA networks.

The models were trained on the ITHACA lab's IEC 60870-5-104 Intrusion Detection Dataset:

<https://zenodo.org/records/7108614>

The project is separated into two parts:

1. *Supervised Learning with boosted-tree algorithms*
2. *Anomaly Detection algorithms*

Supervised Learning Version

The classes of the dataset are:

Class	IEC 104 command	TypeID
NORMAL	Normal traffic	-
c_ci_na_1	Counter Interrogation	101
c_ci_na_1_DoS	Counter Interrogation (DoS)	101
c_rd_na_1	Read Command	102
c_rd_na_1_DoS	Read Command (DoS)	102
c_rp_na_1	Reset Process Command	105
c_rp_na_1_DoS	Reset Process Command (DoS)	105
c_sc_na_1	Single Command	45
c_sc_na_1_DoS	Single Command (DoS)	45
c_se_na_1	Set-point Command	48

Class	IEC 104 command	TypeID
c_se_na_1_DoS	Set-point Command (DoS)	48
m_sp_na_1_DoS	Single-point Information (DoS)	1

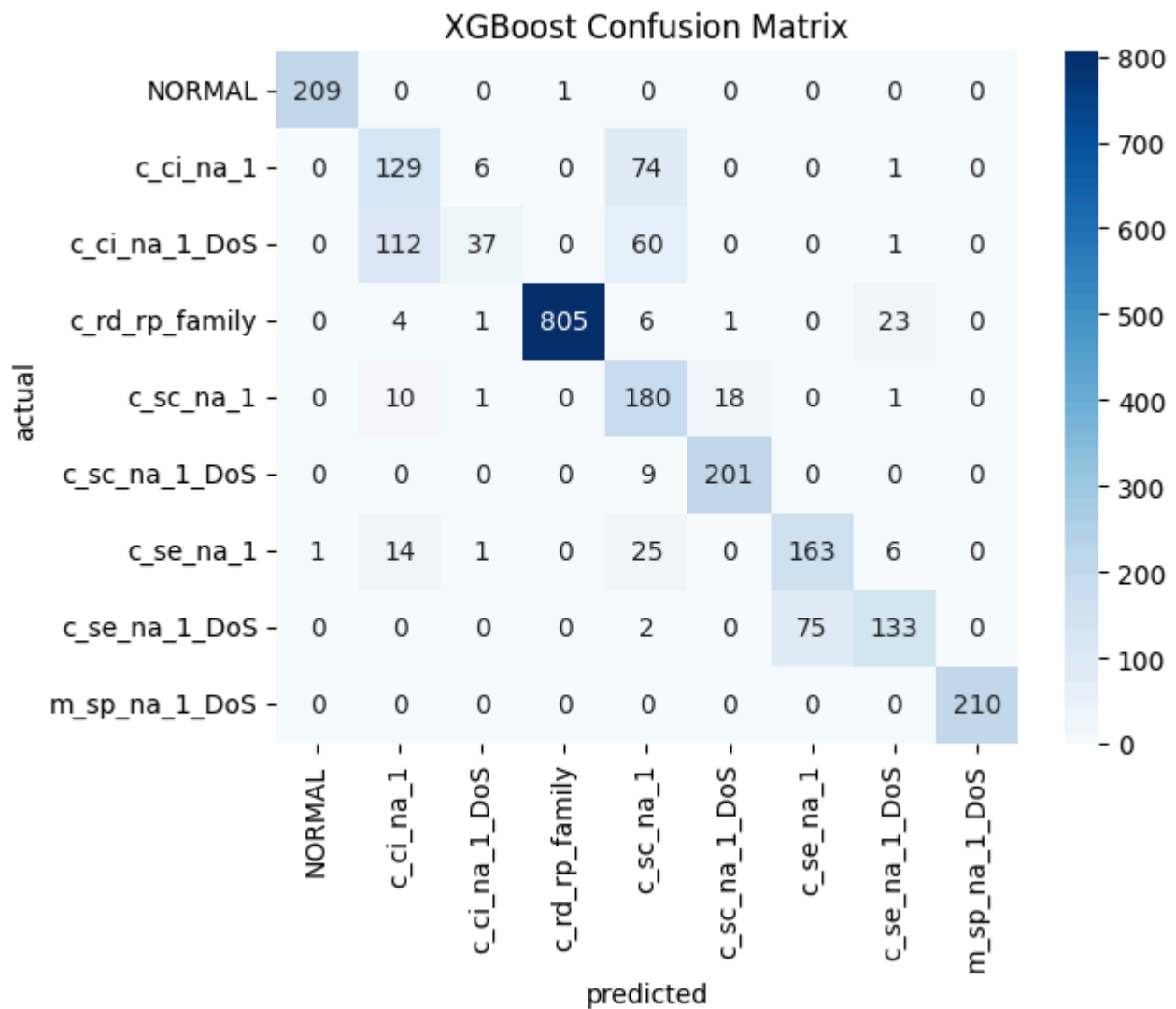
XGBoost Predictions — all 12 classes (accuracy 0.60)

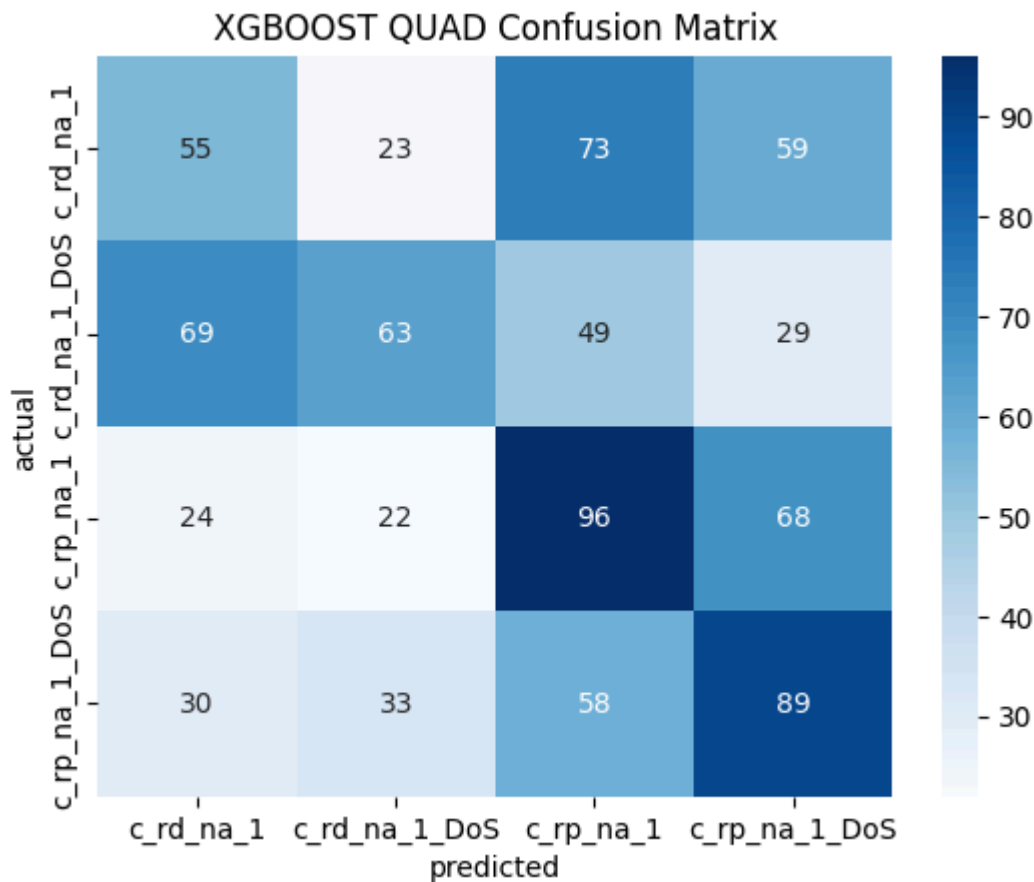
Class	Precision	Recall	F1	Support
NORMAL	1.00	1.00	1.00	210
c_ci_na_1	0.47	0.60	0.53	210
c_ci_na_1_DoS	0.86	0.18	0.30	210
c_rd_na_1	0.18	0.13	0.15	210
c_rd_na_1_DoS	0.49	0.21	0.29	210
c_rp_na_1	0.33	0.44	0.38	210
c_rp_na_1_DoS	0.36	0.50	0.42	210
c_sc_na_1	0.47	0.76	0.58	210
c_sc_na_1_DoS	0.83	0.95	0.89	210
c_se_na_1	0.68	0.78	0.73	210
c_se_na_1_DoS	0.80	0.62	0.70	210
m_sp_na_1_DoS	1.00	1.00	1.00	210
Macro avg	0.62	0.60	0.58	2520
Weighted avg	0.62	0.60	0.58	2520

To understand why the supervised tree-based model plateau, their performance was analysed and visualized with Confusion Matrices, SHAP, ROC-AUC, PR-AUC, calibration curves and UMAP

The confusion matrices of Full and Quad XGB model

As seen in the full model's confusion matrix there's a big misclassification on classes `ci_ci_na_1` and `ci_ci_na_1_DoS` and The other classes are separated more easily because their behavior is more distinctive — for example, sending an ON/OFF command (`c_sc_na_1`) or setting a set-point (an arithmetic value) on a device (`c_se_na_1`).





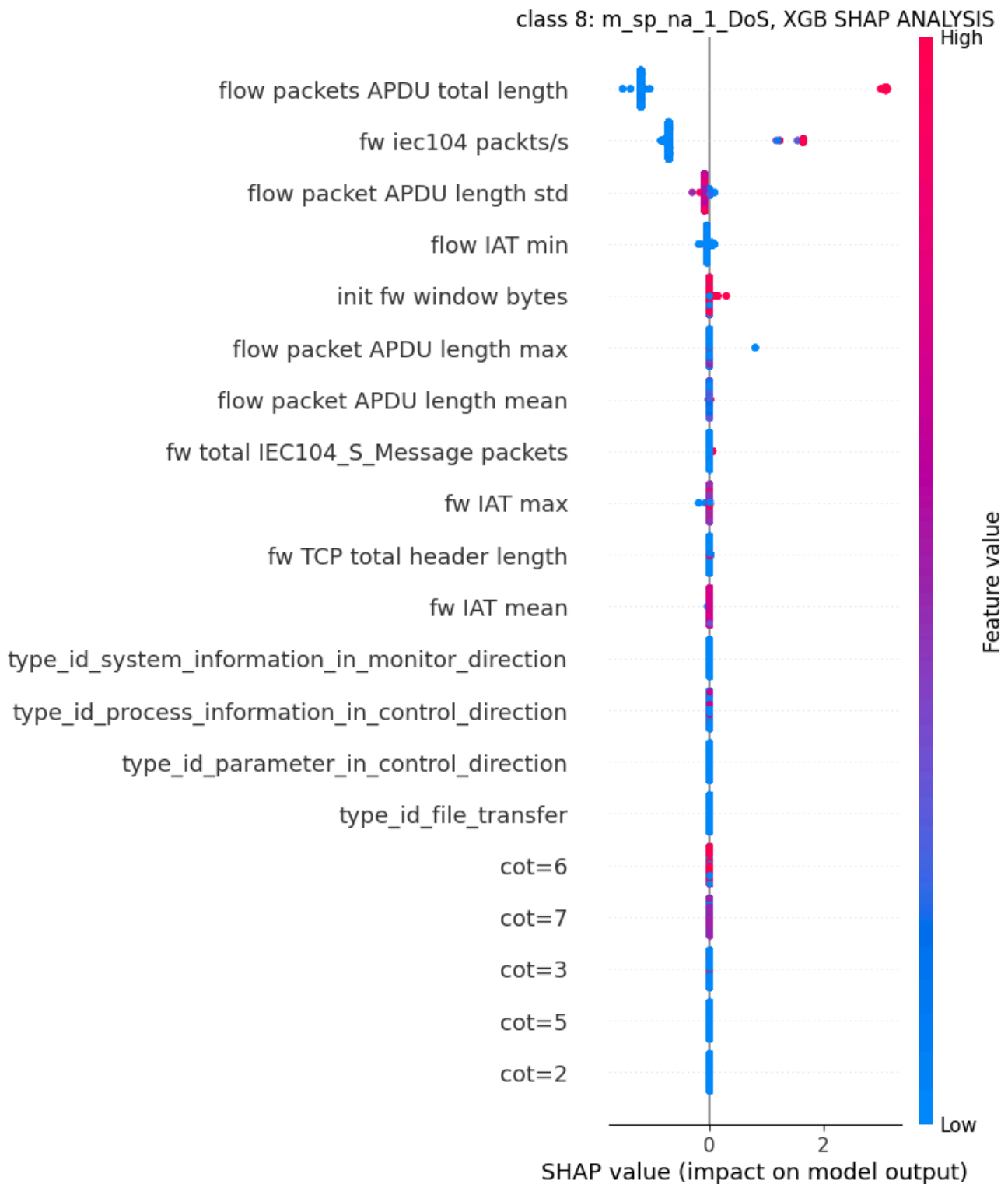
The quad model is underperforming on it's predictions the `c_rd_na_1` and `c_rd_na_1_DoS` is not confusing that much with the `c_rp_na_1` classes
 But there is a conflict between the reset process command classes with them being more misclassified with the read command classes more frequently

SHAP Values

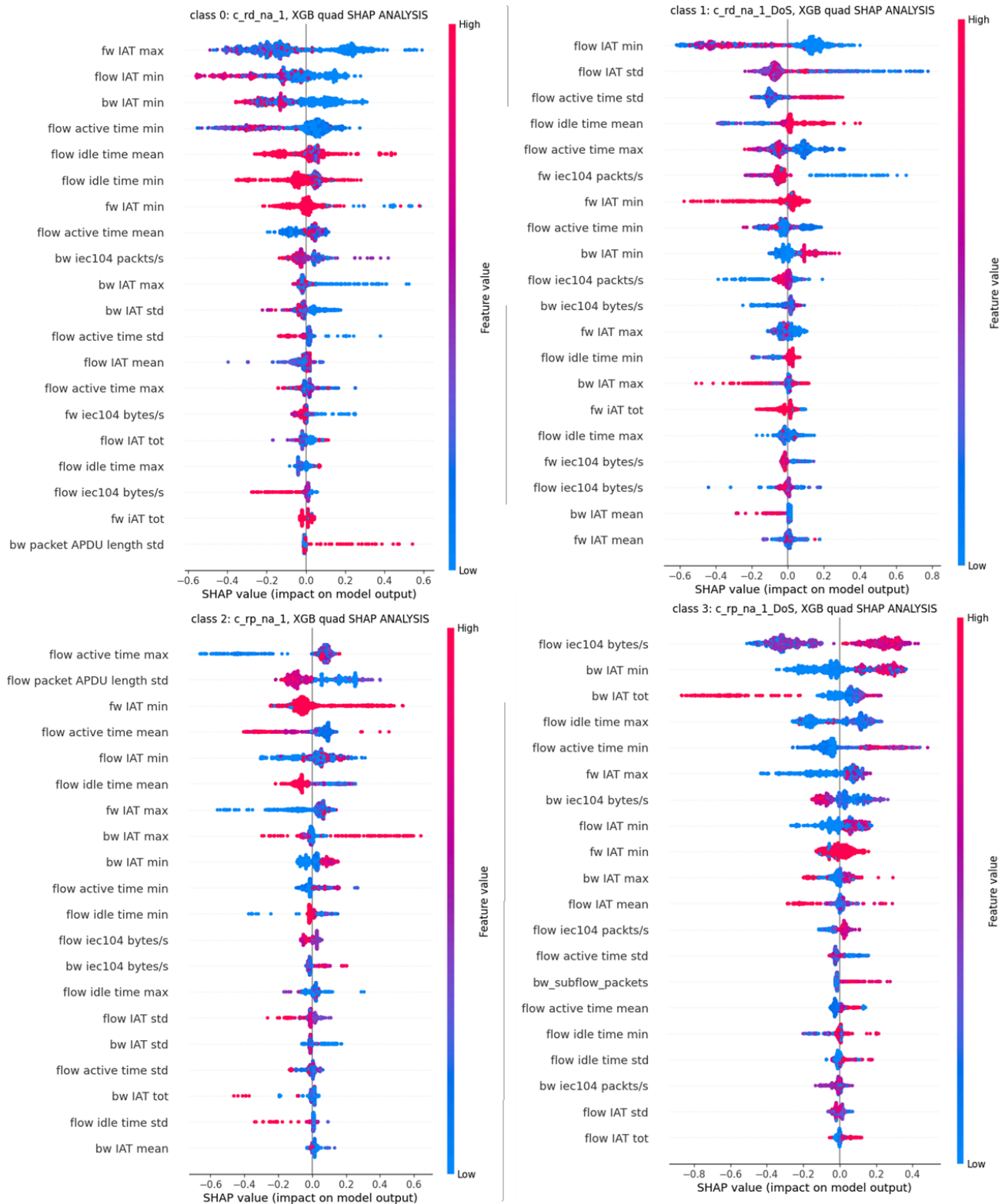
The `NORMAL` Class feature with the most SHAP value is `flow total iec104_U_Message packets` , which makes sense since this feature is built from normal traffic.



For the `m_sp_na_1_DoS` the feature flow packet APDU and fw iec104 packets/s have positive SHAP values that help the model recognize this class, which makes sense since it carries minimal payload (a single binary status point), resulting in a minimal payload per APDU.



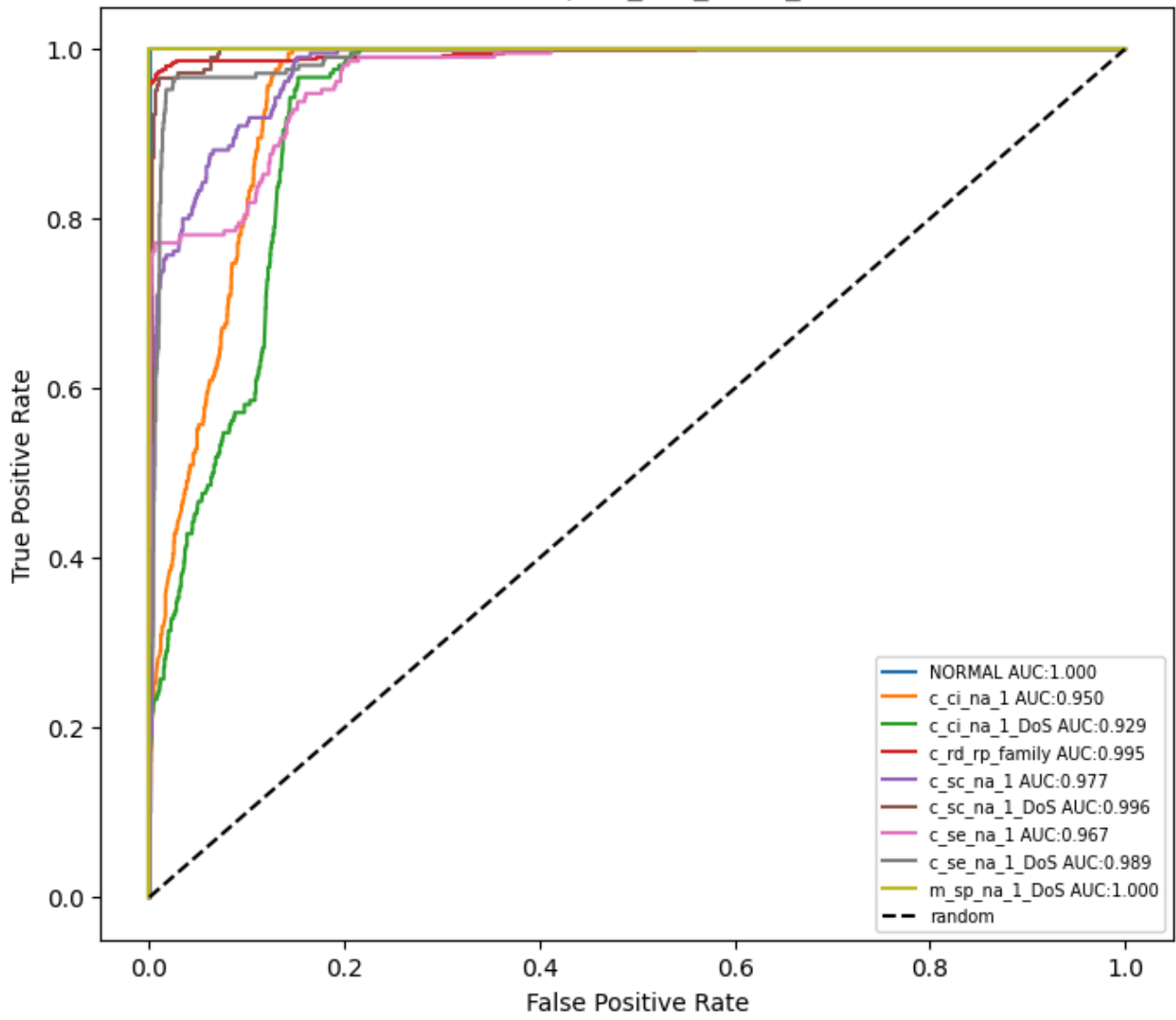
As for the Quad model it's SHAP values for each class *are very small* and similar (roughly -0.6 to 0.6), which makes these classes very hard for the models to separate.



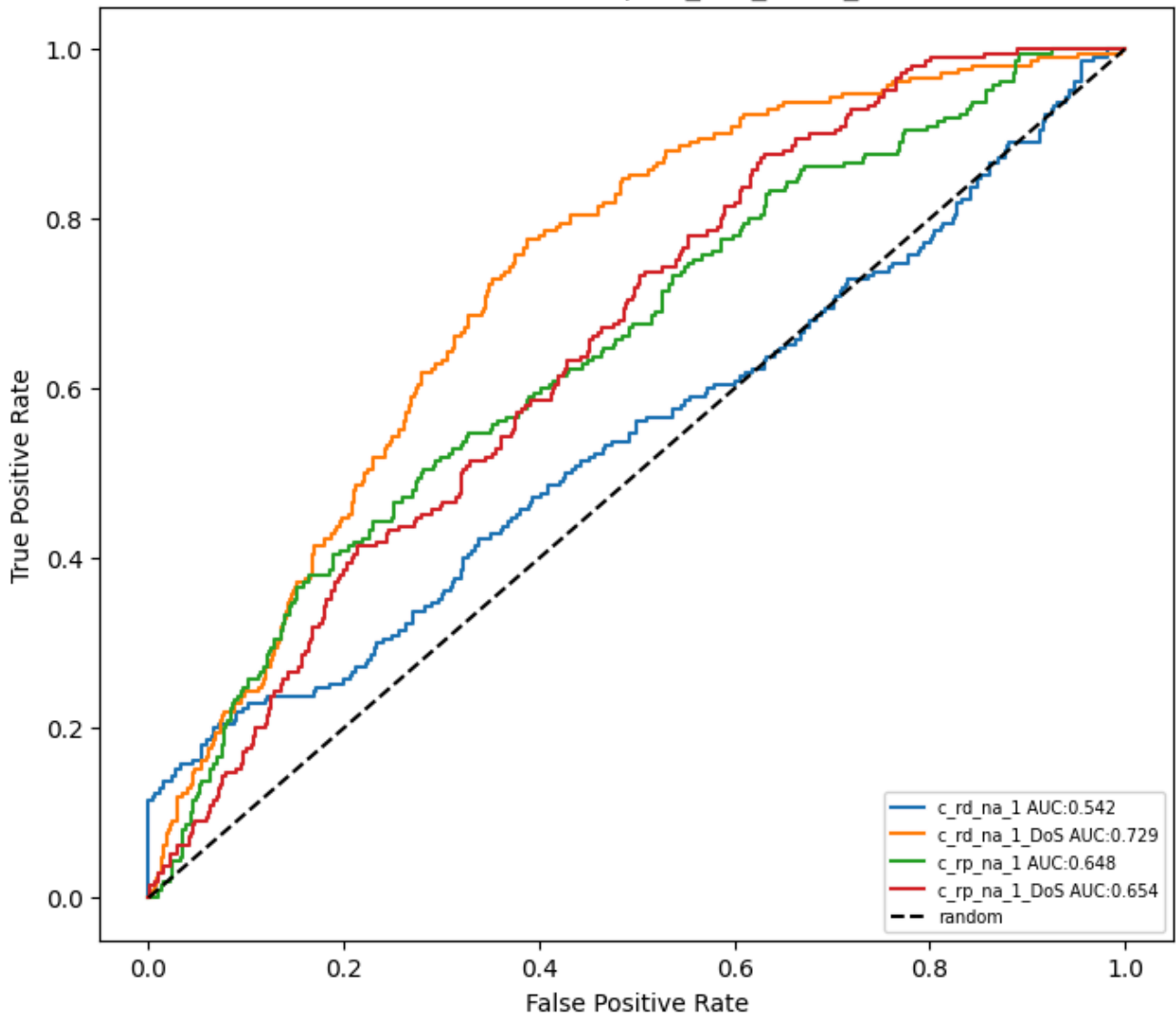
ROC curves

The full-class model reaches good ROC-AUC scores, but the quad models predictions are near random (≈ 0.5).

XGBoost ROC CURVE | roc_auc_score_macro : 0.9781

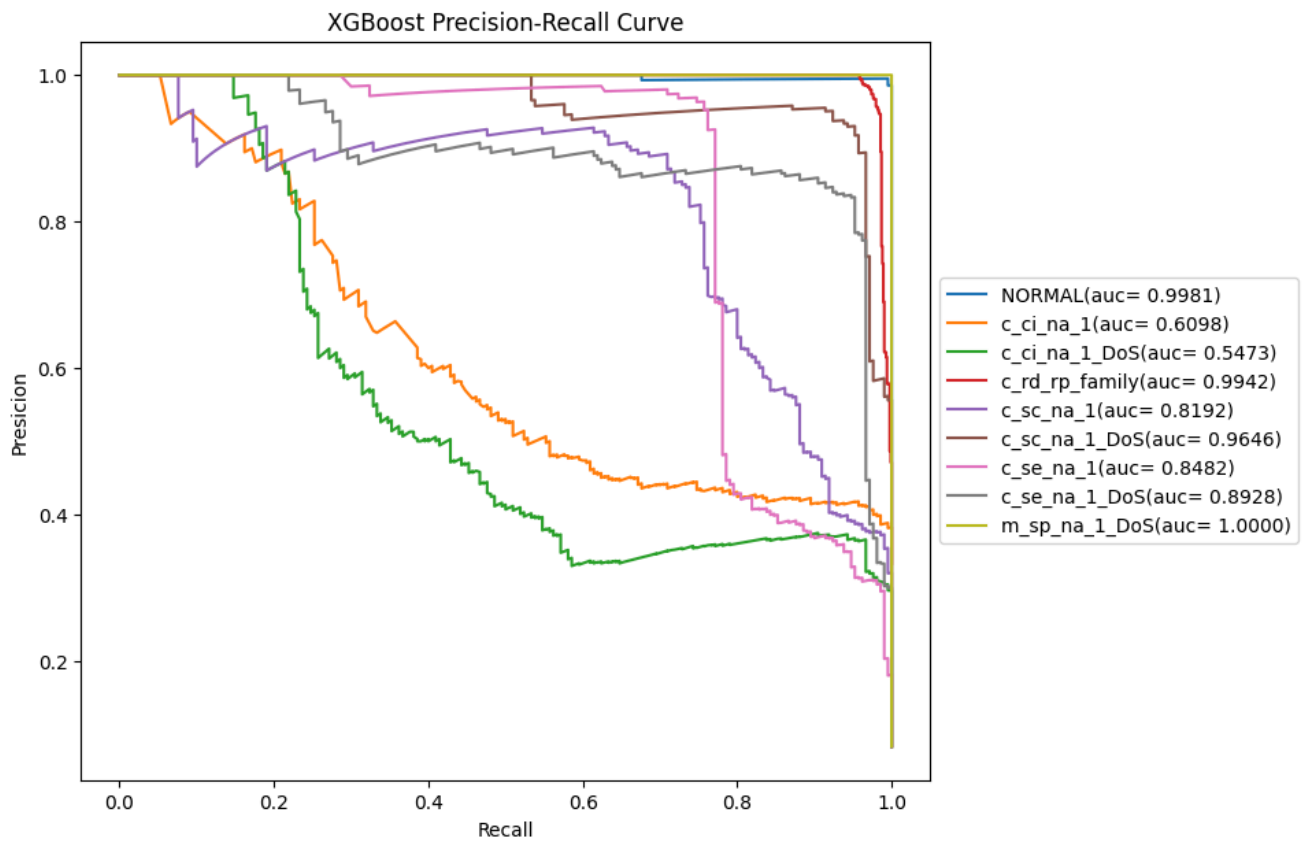


XGBOOST QUAD ROC CURVE | roc_auc_score_macro : 0.6434

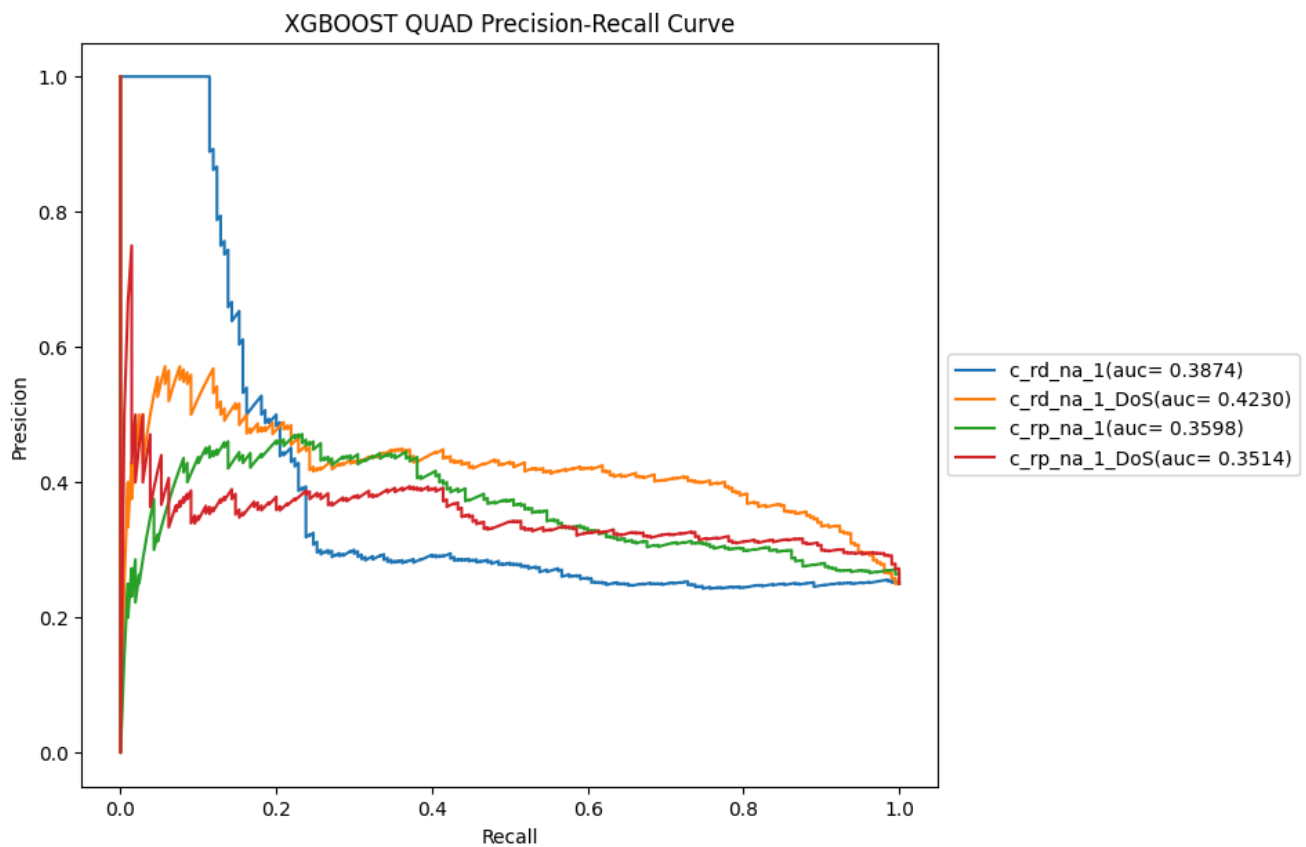


The real AUC score is from the PR-AUC because ROC curve is also adding the true negatives in it's final score

The full XGB model performs very well on normal (0.99), c_rd_rp_family (0.99), c_sc_na_1 (0.81), c_sc_na_1_DoS (0.96), c_se_na_1 (0.84), c_se_na_1_DoS (0.89) and m_sp_na_1_DoS (1.00). The weakest are c_ci_na_1 and c_ci_na_1_DoS, with values near at 0.57

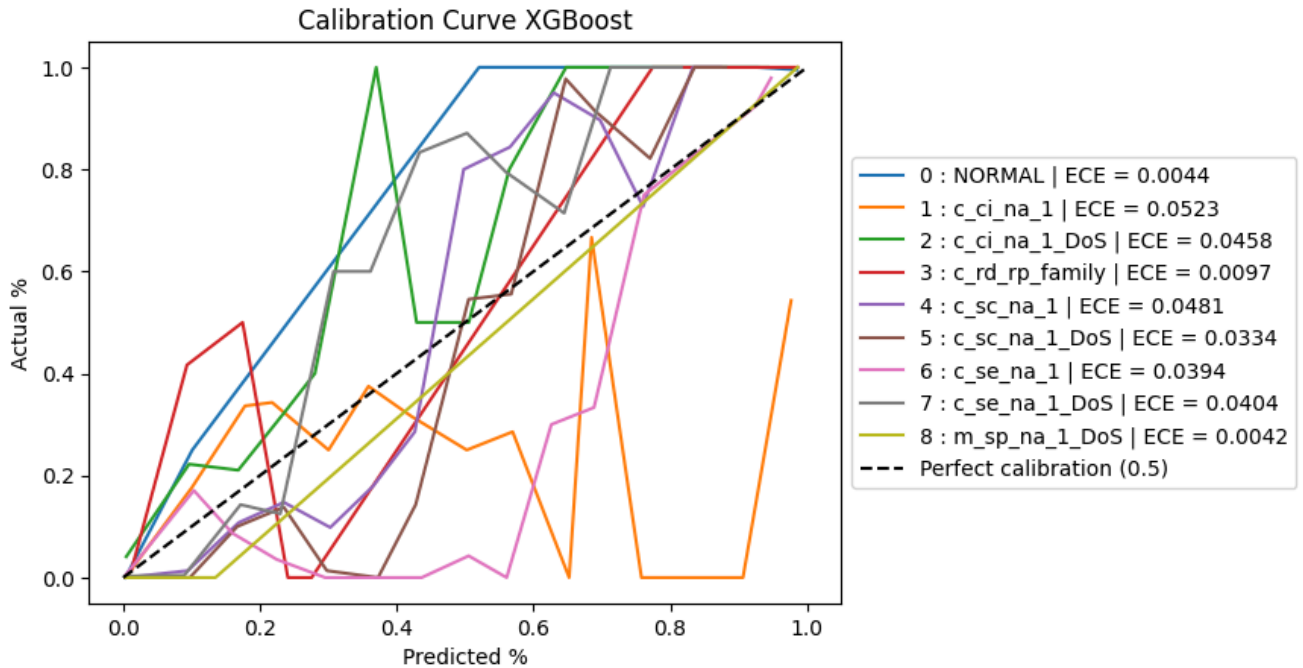


The quad XGB model scores very low, with every class around ~ 0.37 — the model is drowning in false positives and false negatives for these classes.

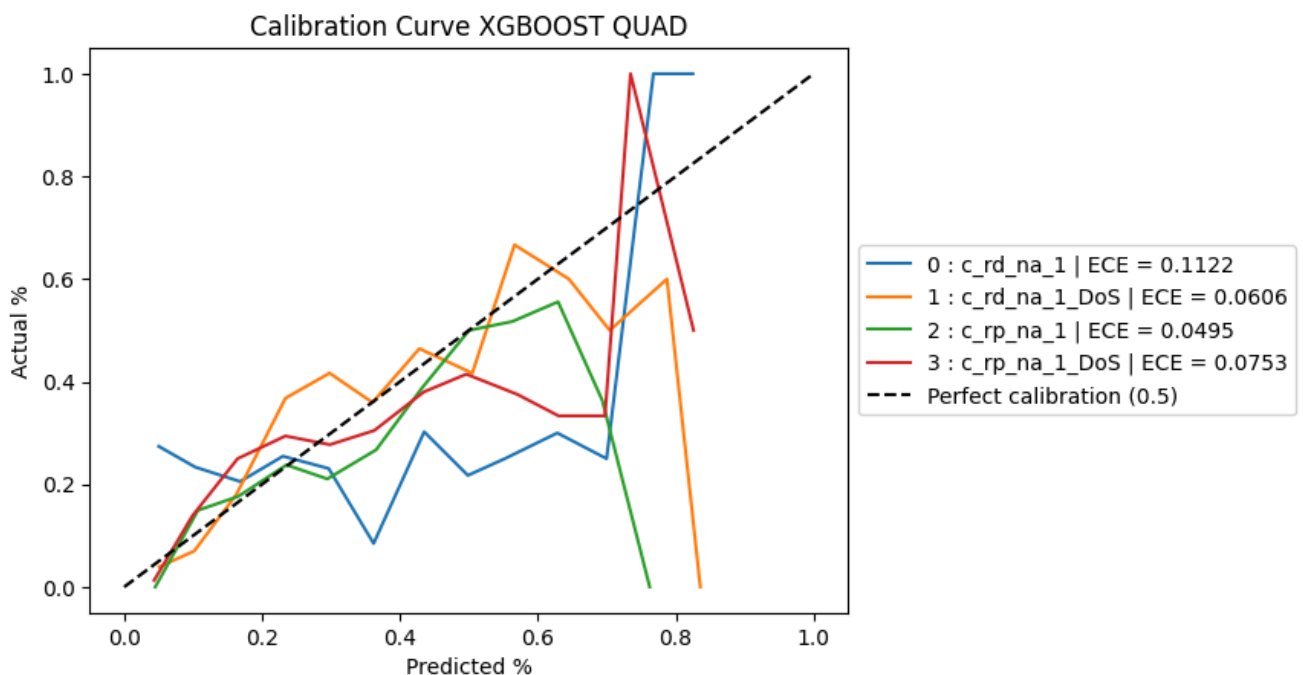


Calibration Curve and Calibrated Models

The XGBOOST model underestimates the DoS variants in parts of the curve and overestimate their normal counterparts.

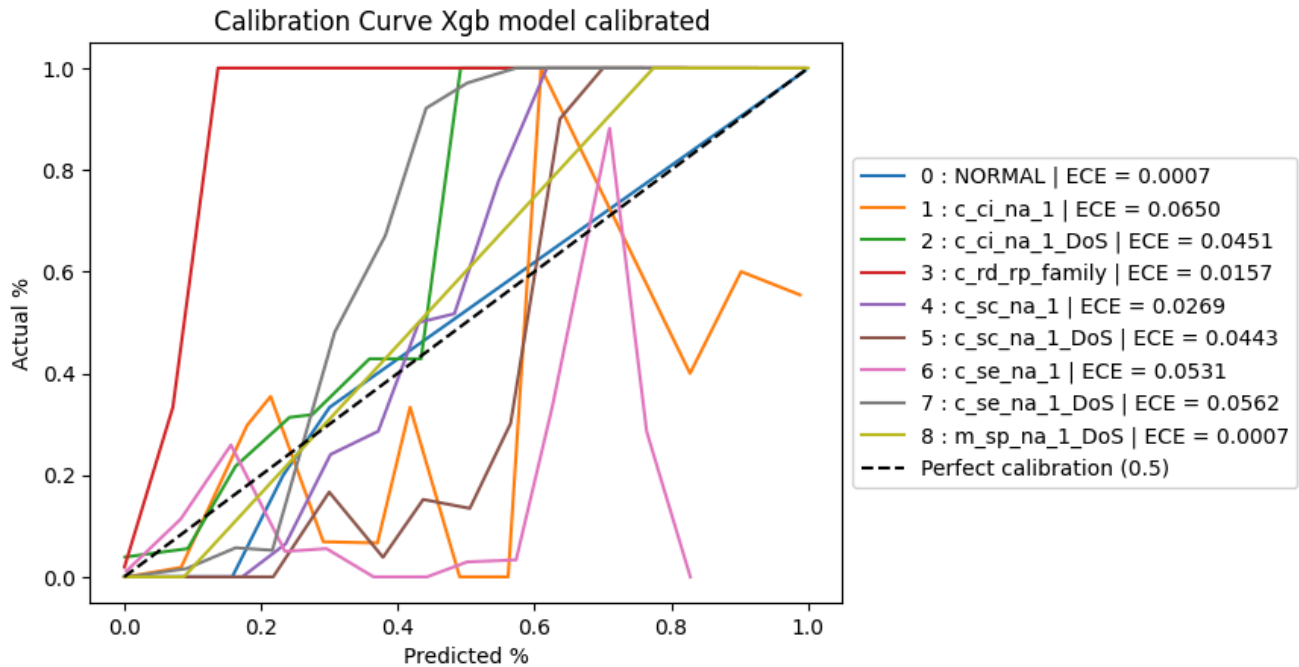


In the quad model, c_rd_na_1_DoS is the most underestimated class and c_rd_na_1 the most overestimated; c_rp_na_1 and c_rp_na_1_DoS behave the same way.



For the full-class calibrated XGB model there is no improvement which means that the model is already well calibrated and it can't do much more, more features are needed to

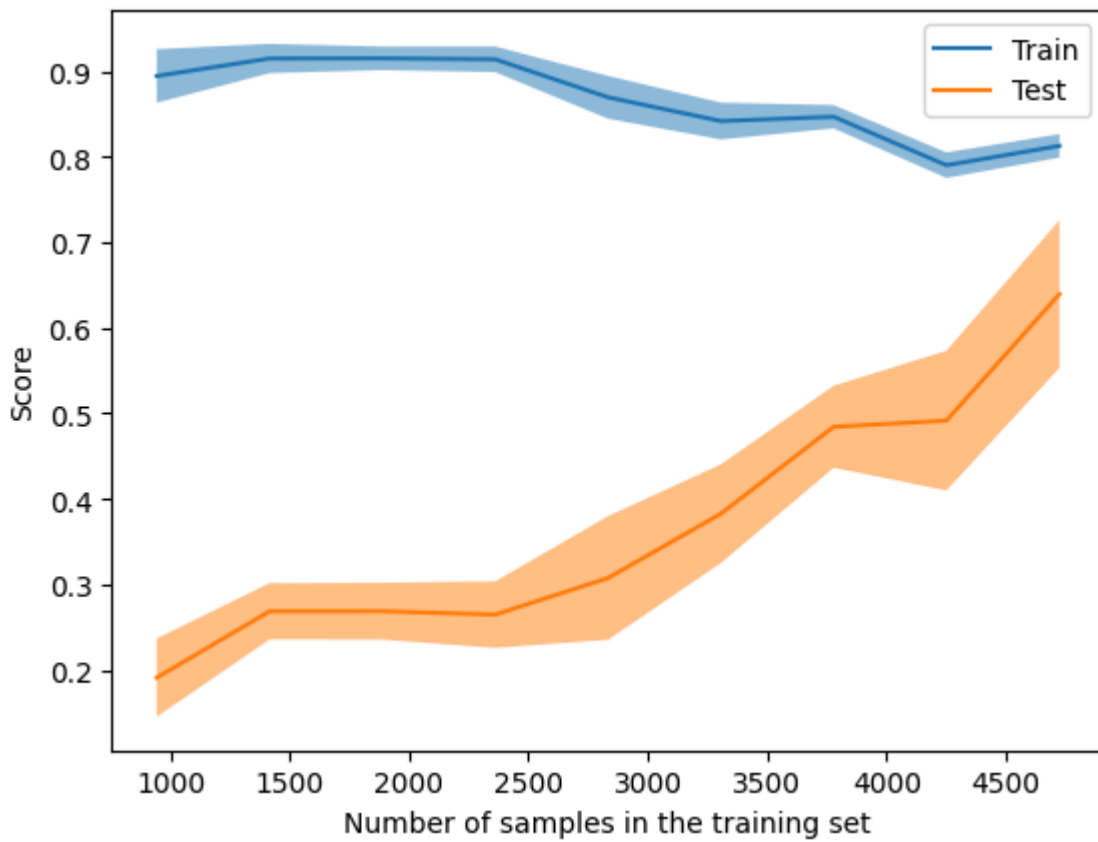
improve the model.



For the XGB Quad model calibration helps a little. Here isotonic calibration improved ECE across all four classes (c_rd_na_1 0.1122 -> 0.0783, c_rd_na_1_DoS 0.0606 -> 0.0541, c_rp_na_1 0.0495 -> 0.0270, c_rp_na_1_DoS 0.0753 -> 0.0560). The features do not separate `c_rd` from `c_rp`, and no amount of calibration can fix that.

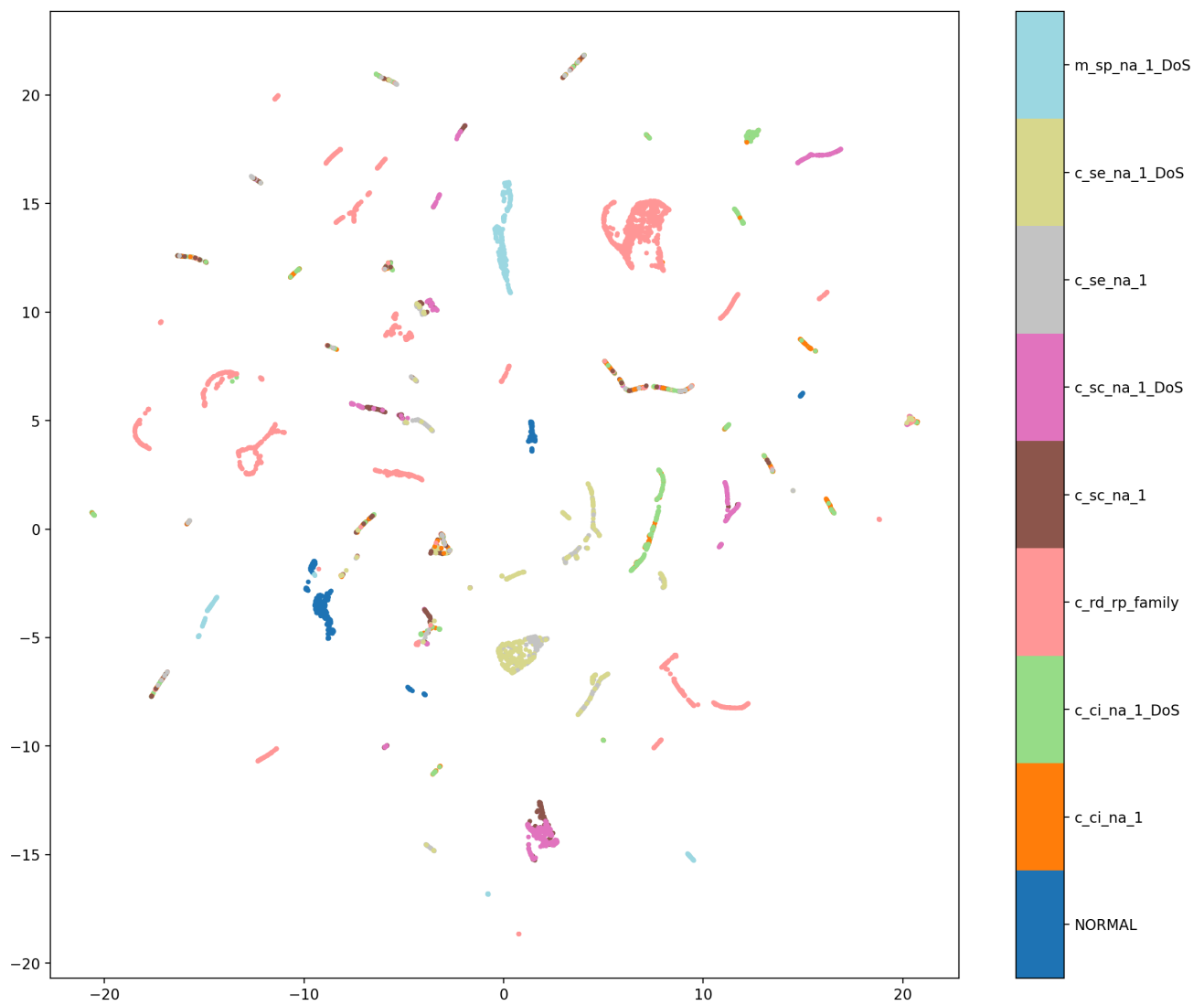
Learning Curve

The Full model as shown from the learning curve as the number of samples progresses it improves it's f1 score so that means that more data maybe could help the model

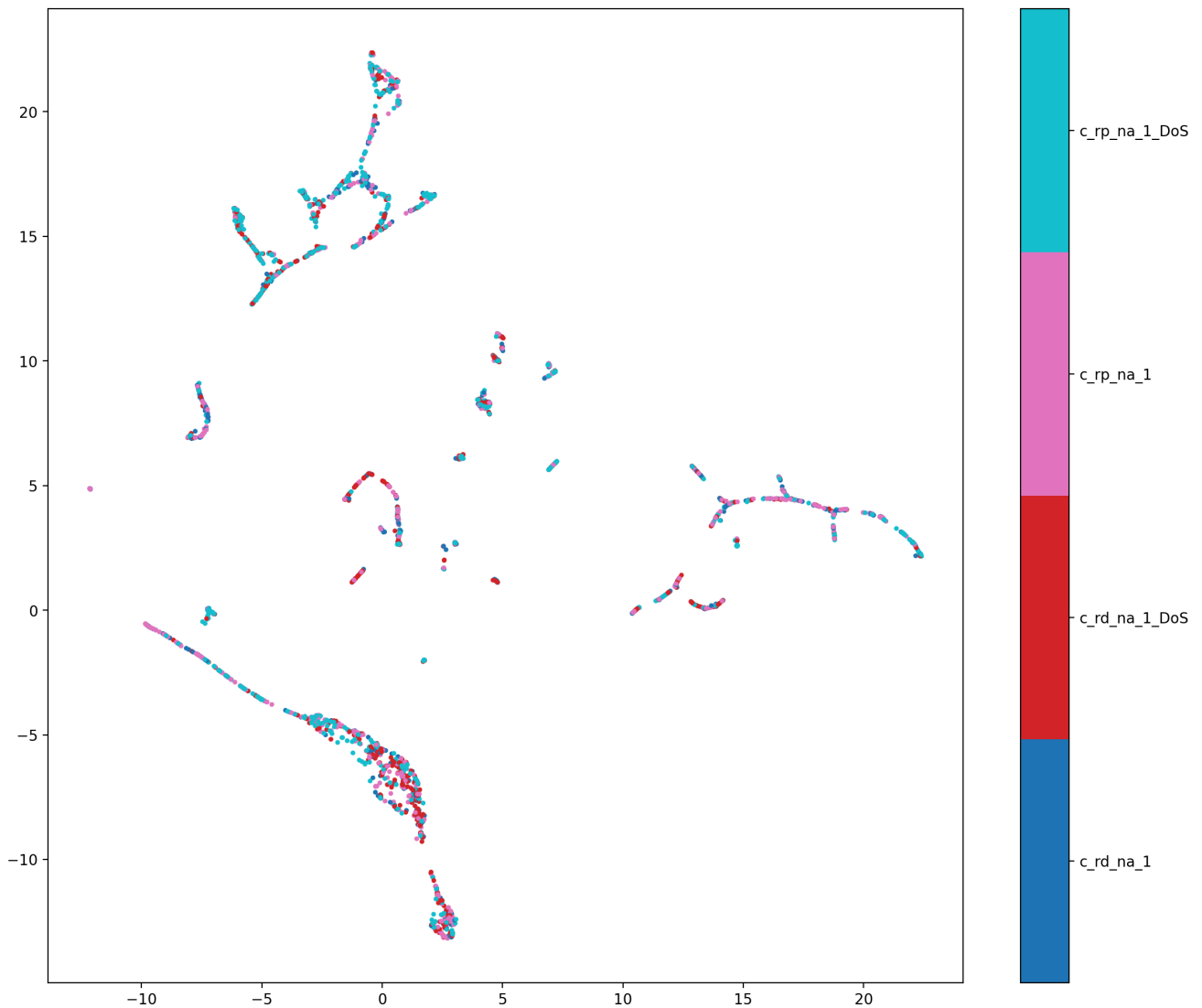


UMAP

In the UMAP plot of the full feature space, every class overlaps with its own DoS counterpart normal and DoS variants of the same command land in the same regions. The most merged and least separated pair is `c_ci_na_1` and its DoS version.



As for the quad family, the four classes ($c_{rd_na_1}$, $c_{rp_na_1}$ and their DoS variants) sit together throughout the projection, confirming that their features have very similar values.



From all the different data/model visualizations we understand that the problem isn't the models themselves but the data from some classes need more information to be right detected.

A two-stage classifier was added, with a model specifically trained on the classes that did worse (c_rd_na_1, c_rd_na_1_DoS, c_rp_na_1, c_rp_na_1_DoS) so that it specializes in predicting only them.

GridSearchCV and RFECV were used to find the best hyperparameters of all models and drop the unnecessary features.

Results of the Supervised Learning version

Stage 1 — XGBoost, 9 classes, accuracy 0.82, macro f1 0.76:

class	precision	recall	f1	support
NORMAL	1.00	1.00	1.00	210
c_ci_na_1	0.48	0.61	0.54	210
c_ci_na_1_DoS	0.80	0.18	0.29	210
c_rd_rp_family	1.00	0.96	0.98	840
c_sc_na_1	0.51	0.86	0.64	210
c_sc_na_1_DoS	0.91	0.96	0.93	210
c_se_na_1	0.68	0.78	0.73	210
c_se_na_1_DoS	0.81	0.63	0.71	210
m_sp_na_1_DoS	1.00	1.00	1.00	210
macro avg	0.80	0.77	0.76	2520
weighted avg	0.85	0.82	0.81	2520

Stage 2 — quad XGB model, 4 classes, accuracy 0.36, macro f1 0.35:

class	precision	recall	f1	support
c_rd_na_1	0.24	0.19	0.21	181
c_rd_na_1_DoS	0.46	0.32	0.38	204
c_rp_na_1	0.36	0.45	0.40	210
c_rp_na_1_DoS	0.37	0.45	0.41	210
macro avg	0.36	0.35	0.35	805
weighted avg	0.36	0.36	0.35	805

The model lost 29 samples from c_rd_na_1 and 6 samples from c_rd_na_1_DoS, which means the first-stage classifier couldn't even recognize those samples as the

c_rd_rp_family .

Comparison

class	single-stage XGB	two-stage specialist	difference f1
c_rd_na_1	0.15	0.21	+0.06
c_rd_na_1_DoS	0.29	0.38	+0.09
c_rp_na_1	0.38	0.40	+0.02
c_rp_na_1_DoS	0.42	0.41	-0.01

The gain was only in the read command classes and the reset process command was almost none.

Anomaly Detection

Anomaly Detection Algorithms are more helpful on IDS systems since they can provide protection from **zero-day** attacks and they just have to know how the Normal traffic is

Models used:

- Isolation Forest
- One class Support Vector Machine
- Autoencoder (PyTorch)

Results of the Anomaly Detection Version

Isolation Forest — accuracy 0.9980 (contamination 0.02, n_estimators 100)

class	precision	recall	f1	support
Attack	1.00	1.00	1.00	2310
Normal	1.00	0.98	0.99	210

class	precision	recall	f1	support
macro avg	1.00	0.99	0.99	2520
weighted avg	1.00	1.00	1.00	2520

One-Class SVM — accuracy 0.9980 (nu 0.01, gamma 0.001)

class	precision	recall	f1	support
Attack	1.00	1.00	1.00	2310
Normal	1.00	0.98	0.99	210
macro avg	1.00	0.99	0.99	2520
weighted avg	1.00	1.00	1.00	2520

Autoencoder — accuracy 0.9988 (epochs 1000)

class	precision	recall	f1	support
Attack	1.00	1.00	1.00	2310
Normal	1.00	0.99	0.99	210
macro avg	1.00	0.99	0.99	2520
weighted avg	1.00	1.00	1.00	2520

As seen in the plot of Training Loss of the Autoencoder, as the number of epochs progresses the reconstruction loss of the normal samples reduces



All the models do very well on detecting the attacks and the normal classes.

Conclusion

The Anomaly detection methods were almost 100% accurate while the Supervised version is dealing with classes that look very similar to each other because of their features. The anomaly detection method is more useful on a IDS since the main goal is to identify the attack rather than classify it

To further improve the Supervised Learning Version more features are needed, by this way we can take different measures for each attack type in the SCADA Networks

Dataset & Citation

This project uses the **IEC 60870-5-104 Intrusion Detection Dataset**, created by the ITHACA lab at the University of Western Macedonia (<https://ithaca.ece.uowm.gr/>).

- Zenodo record: <https://zenodo.org/records/7108614>
- DOI: [10.21227/fj7s-f281](https://doi.org/10.21227/fj7s-f281)

- License: [CC BY 4.0](#)

Authors: P. Radoglou-Grammatikis, K. Rompolos, T. Lagkas, V. Argyriou, P. Sarigiannidis

As requested by the dataset authors, please cite:

P. Radoglou-Grammatikis, K. Rompolos, P. Sarigiannidis, V. Argyriou, T. Lagkas, A. Sarigiannidis, S. Goudos and S. Wan, "Modeling, Detecting, and Mitigating Threats Against Industrial Healthcare Systems: A Combined Software Defined Networking and Reinforcement Learning Approach," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 3, pp. 2041–2052, March 2022, doi: 10.1109/TII.2021.3093905.
<https://ieeexplore.ieee.org/document/9470933>

The dataset was produced within the H2020 projects **ELECTRON** (grant agreement No 101021936) and **SDN-microSENSE** (No 833955).

The IEC 60870-5-104 flow statistics used here come from the dataset's Custom IEC 60870-5-104 Python Parser (Scapy-based); the TCP/IP flow statistics come from CICFlowMeter.